

DOCUMENTACIÓN TÉCNICA

Sistema de Gestión de Reservas Hoteleras

fundReserves — Prueba Técnica Analista Desarrollador .NET

Tecnología: ASP.NET Core MVC 8 · C# · Entity Framework Core · SQL Server

Repositorio: github.com/jhoancamilorayomejia/fundReserves

1. Introducción

El presente documento describe el desarrollo del Sistema de Gestión de Reservas Hoteleras (fundReserves), una solución diseñada para el Fondo XYZ con el objetivo de optimizar la administración de alojamientos, reservas y disponibilidad en tiempo real dentro de sus sedes recreativas y apartamentos ubicados en distintas ciudades del país.

El sistema responde a la necesidad de ofrecer a los asociados y su grupo familiar una plataforma centralizada que permita:

- Consultar cada uno de los lugares disponibles con sus características.
- Ver la disponibilidad del sitio (sedes recreativas y apartamentos) de acuerdo a la fecha de viaje.
- Ver las tarifas de acuerdo al sitio, la temporada, el número de personas y el alojamiento elegido.
- Realizar el cálculo de la tarifa a cancelar según el sitio, número de habitaciones, personas, tipo de alojamiento y temporada.
- Ejecutar y gestionar reservas de forma segura, con control de sesiones por rol.

Adicionalmente, proporciona a los administradores herramientas de gestión completa sobre alojamientos, tarifas, temporadas y reservas, mejorando la organización operativa y reduciendo errores asociados a procesos manuales.

2. Arquitectura del Sistema

2.1 Arquitectura Cliente–Servidor

El sistema fue desarrollado bajo una arquitectura Cliente–Servidor con las siguientes tecnologías principales:

Capa	Tecnología
Backend	ASP.NET Core MVC (.NET 8) con C#
ORM	Entity Framework Core 8.x
Base de datos	Microsoft SQL Server 2019+
Frontend	Razor Views + HTML5 + CSS3 + JavaScript
Seguridad	Sesiones ASP.NET Core (Cookie HttpOnly) + Hash de contraseñas
Correo	SMTP Gmail (System.Net.Mail)

2.2 Patrón Arquitectónico: MVC + Programación en Capas

Se implementó el patrón Model-View-Controller (MVC) combinado con programación en capas, lo que permite:

- Separación de responsabilidades entre presentación, lógica de negocio y acceso a datos.
- Escalabilidad: cada capa puede evolucionar de forma independiente.
- Facilidad de mantenimiento y pruebas unitarias por capa.
- Reutilización de componentes (ej. EmailService consumible desde cualquier controlador).

2.3 Flujo General del Sistema

Usuario Cliente:

1. El usuario accede al sistema mediante autenticación válida del rol (Admin / Customer), (login con sesión) y se redirige al dashboard correspondiente
2. El usuario accede a la sede a la cual quiere hacer la reserva.
3. El cliente consulta la disponibilidad indicando fechas y número de personas.
4. El backend invoca el procedimiento almacenado GetAvailable en SQL Server.
5. SQL Server ejecuta la lógica de cruce de fechas y retorna alojamientos disponibles.
6. El cliente selecciona el alojamiento; el sistema invoca el SP de cálculo de tarifa.
7. El total calculado se presenta al cliente para confirmación.
8. El cliente hace la reserva del alojamiento (solo si esta disponible)
9. El Administrador puede consultar, cambiar estado o eliminar reservas desde su panel.

Usuario Administrador:

1. El usuario accede al sistema mediante autenticación válida del rol (Admin / Customer), (login con sesión) y se redirige al dashboard correspondiente
2. El usuario accede a visualizar las reservas creadas por los clientes y verificar el estado y el comprobante subido.
3. El usuario podrá visualizar, crear, actualizar y eliminar sedes, alojamientos, temporadas, tarifas.

3. Estructura de Capas del Proyecto

3.1 Árbol de Directorios

```
FoundReserves/
├── Controllers/                ← Reciben HTTP
│   ├── HomeController.cs      ← Login, logout
│   ├── AuthController.cs     ← Registro y gestión de usuarios
│   ├── UseController.cs      ← Dashboard admin y perfil
│   ├── sedesController.cs    ← CRUD sedes
│   ├── seasonController.cs   ← CRUD temporadas
│   ├── ratesController.cs    ← CRUD tarifas
│   ├── reservesController.cs ← CRUD reservas por alojamiento
│   └── DetalleReserveController.cs ← CRUD reservas por sedes
├── Models/                   ← Entidades mapeadas a BD (EF Core)
│   ├── Accommodation.cs
│   ├── Auth.cs (= User.cs)
│   ├── DetalleReserve.cs
│   ├── EmailSettings.cs
│   ├── rates.cs
│   ├── reserves.cs
│   ├── season.cs
│   └── Sede.cs
├── Data/
│   └── ApplicationDbContext.cs ← Contexto EF Core, DbSets, relaciones
└── Services/
    └── IEmailService.cs      ← Interfaz del servicio de email
```

```

├── EmailService.cs      ← Implementación SMTP Gmail
├── ViewModels/ChangePasswordViewModels.cs      ← DTOs para vistas que combinan
múltiples entidades
├── Views/
│   ├── Customer/
│   │   ├── DashboardCustomer.cshtml      ← Panel del cliente
│   │   └── reservesCustomer.cshtml      ← Formulario de reservas
│   ├── Home/
│   │   ├── Index.cshtml      ← Login
│   │   ├── ForgotPassword.cshtml      ← Recuperación contraseña
│   │   ├── EmailSent.cshtml      ← Confirmación envío enlace
│   │   └── ChangePassword.cshtml      ← Nueva contraseña
│   ├── Dashboard.cshtml      ← Panel administrador
│   └── Shared/
│       └── _AccountLayout.cshtml      ← Layout recuperación contraseña
├── wwwroot/
│   ├── css/      (login, dashboard, dashboardCustomer, reservesCustomer, site)
│   ├── images/      (Bogota, Chinchina, fusagasuga, Medellin, palmira...)
│   └── uploads/comprobantes/      ← Archivos subidos por clientes
├── Program.cs      ← Punto de entrada, DI, middlewares
├── appsettings.json      ← Configuración conexión y SMTP
└── FoundReserves.csproj      ← Dependencias NuGet (.NET 8)

```

3.2 Descripción de Cada Capa

Controllers

Reciben solicitudes HTTP, orquestan la interacción entre modelos y vistas, validan datos de entrada y retornan vistas Razor o respuestas JSON. Cada controlador agrupa las operaciones relacionadas con un dominio del negocio.

Models

Representan las entidades de la base de datos mediante clases C# mapeadas con Entity Framework Core. Definen propiedades, tipos de datos y anotaciones de validación ([Required], [MaxLength], etc.).

Data — ApplicationDbContext

Contexto de Entity Framework Core que configura la conexión a SQL Server, define los DbSet<T> por entidad, establece las relaciones y gestiona las migraciones de esquema.

Services (modificar)

Encapsula lógica de negocio reutilizable desacoplada de los controladores. El servicio EmailService implementa la interfaz IEmailService y se registra en el contenedor DI como AddTransient, permitiendo inyección en cualquier controlador.

ViewModels (modificar)

Data Transfer Objects (DTOs) que combinan datos de múltiples entidades para ser consumidos por las vistas Razor, evitando exponer directamente los modelos de BD al frontend.

Views

Plantillas Razor (.cshtml) que generan HTML dinámico combinando C# con marcado. Consumen los ViewModels o los modelos directamente para renderizar formularios, tablas y paneles según el rol del usuario.

4. Rutas HTTP por Controlador

El sistema sigue la convención de rutas MVC de ASP.NET Core: `/controller/action/{id?}`. La ruta por defecto apunta a `Home/Index` (formulario de login).

4.1 HomeController — Autenticación y Recuperación de Contraseña

Método	Ruta	Descripción
GET	/Home/Index	Muestra formulario de login (vista Razor)
POST	/Home/Login	Autentica con BCrypt; persiste 7 valores en Session (IdUser, Email, Rol, Cedula, Name, Lastname, Phone); redirige a Dashboard o DashboardCustomer según rol
GET	/Home/Dashboard	Panel del Admin; protegido manualmente verificando sesión y rol "Admin"
GET	/Home/DashboardCustomer	Panel del cliente; protegido verificando sesión y rol "Customer"; renderiza /Views/Customer/DashboardCustomer.cshtml
GET	/Home/ReservesCustomer?idSede=	Vista del formulario de reservas del cliente; acepta idSede opcional vía query string; protegido por sesión
GET	/Home/ForgotPassword	Muestra formulario para ingresar correo de recuperación
POST	/Home/ForgotPassword	Genera token GUID, lo persiste con expiración de 1 hora, envía email HTML con enlace vía SMTP; siempre redirige a EmailSent sin revelar si el correo existe
GET	/Home/ChangePassword?token=&email=	Muestra formulario para nueva contraseña; recibe token y email como query params y los pasa al ViewModel
POST	/Home/ChangePassword	Valida token, email y expiración en BD; hashnea nueva contraseña con BCrypt; invalida el token seteándolo a null
GET	/Home/EmailSent	Vista de confirmación de envío del enlace de recuperación

POST	/Home/Logout	Limpia toda la sesión con Session.Clear() y redirige al login
------	--------------	---

4.2 AuthController — Gestión de Usuarios – [Route("api/auth")]

Método	Ruta	Descripción
POST	/api/auth/login	Válida cédula y contraseña por comparación directa (este endpoint es API pura, sin sesión); retorna datos del usuario en JSON
POST	/api/auth/register	Registra nuevo usuario con rol "Customer" forzado; verificando que no hayan duplicados de email y cédula; contraseña con BCrypt

4.3 sedesController — Gestión de Sedes

Método	Ruta	Descripción
GET	/api/sedes/all	Lista todas las sedes ordenadas por nombre (cualquier rol autenticado por sesión)
POST	/api/sedes/create	Crea nueva sede validando nombre y ciudad obligatorios (solo Admin por sesión)
PUT	/api/sedes/update/{id}	Actualiza los campos name, city, region, type, maximumCapacity, priceLaundry y description de una sede por ID (solo Admin)
DELETE	/api/sedes/{id}	Elimina una sede por ID (solo Admin)

4.4 seasonController — Gestión de Temporadas

Método	Ruta	Descripción
GET	/api/seasons/all	Lista todas las temporadas con id, name y type (cualquier rol autenticado)
POST	/api/seasons/create	Crea nueva temporada validando name y type obligatorios (solo Admin)
DELETE	/api/seasons/{id}	Elimina temporada por ID (solo Admin)
PUT	/api/seasons/update/{id}	Actualiza name y/o type de una temporada por ID (solo Admin)

4.5 ratesController — Gestión de Tarifas

Método	Ruta	Descripción
GET	/api/rates/all	Lista tarifas con JOIN a accommodation, seasons y sedes; retorna nombre formateado [Sede - Ciudad] Alojamiento - N° X
DELETE	/api/rates/{id}	Elimina tarifa por ID (solo Admin)
GET	/api/rates/byAccommodation/{idAccommodation}	Retorna tarifas de un alojamiento específico con datos de su temporada, ordenadas por idSeason
PUT	/api/rates/update/{id}	Actualiza minimumPerson , maximumPerson , priceNight y pricePersonAdditional de una tarifa (solo Admin)

4.6 reservesController — Reservas por Alojamiento – [Route(“api/reservations”)]

Método	Ruta	Descripción
POST	/api/reservations/create	Crea reserva; válida cruce de fechas excluyendo estado "Cancelada"; asigna estado "Pendiente" y fecha de creación automáticamente

Método	Ruta	Descripción
GET	/api/reservations/my	Lista reservas del usuario en sesión (idUser de Session), ordenadas por fecha de creación descendente
GET	/api/reservations/byUserAndDate?idUser=&dateStart=&dateFinish=	Filtra reservas por usuario y fechas exactas; retorna con nombre y descripción del alojamiento vía JOIN
GET	/api/reservations/available?idSede=&dateStart=&dateFinish=&persons=	Retorna alojamientos disponibles de una sede, filtrando ocupados por cruce de fechas y capacidad; incluye tarifas aplicables al rango de personas
PUT	/api/reservations/updateState/{idReservation}	Actualiza estado de reserva; acepta solo "Pendiente", "Pagado" o "Cancelado" (cualquier rol autenticado)

4.7 DetalleReserveController — Reserva por Sede Completa

Método	Ruta	Descripción
POST	/api/detalleReserve/create	Crea detalle de reserva por sede completa; paymentProof queda en null hasta que se suba el comprobante
POST	/api/detalleReserve/uploadProof/{id}	Sube comprobante en PDF únicamente; genera nombre único con Guid.NewGuid(); guarda en wwwroot/uploads/comprobantes/ y persiste la ruta relativa en BD
GET	/api/detalleReserve/byUser/{idUser}	Lista detalles de reserva de un usuario, ordenados por id descendente
GET	/api/detalleReserve/all	Lista todos los detalles de reserva sin filtros ni joins
GET	/api/detalleReserve/allWithDetails	Lista detalles con nombre, apellido, cédula del usuario y nombre de la sede via JOIN (solo Admin por sesión)
DELETE	/api/detalleReserve/{id}	Elimina detalle de reserva por ID

4.8 UserController — Panel Administrativo – [Route("api/users")]

Método	Ruta	Descripción
GET	/api/users/all	Lista todos los usuarios sin exponer contraseña, incluyendo datos personales completos (solo Admin por sesión)
DELETE	/api/users/{id}	Elimina usuario por ID físicamente de la BD (solo Admin por sesión)
POST	/api/users/createAdmin	Crea un nuevo usuario con rol Admin fijo; valida correo único y campos obligatorios (solo Admin por sesión)

Método	Ruta	Descripción
PUT	/api/users/update/{id}	Actualiza datos de un usuario por ID; la contraseña solo se modifica si se envía (solo Admin por sesión)

4.9 AccommodationController — [Route(“api/accommodation”)]

Método	Ruta	Descripción
GET	/api/accommodation/all	Lista alojamientos con nombre de sede y ciudad vía Include; cualquier rol autenticado
POST	/api/accommodation/create	Crea alojamiento vinculado a una sede; sin validaciones adicionales en este endpoint
DELETE	/api/accommodation/{id}	Elimina alojamiento por ID; cualquier rol autenticado
PUT	/api/accommodation/update/{id}	Actualiza campos name, number, maximumPerson, state, description e idSede de un alojamiento (solo Admin)

5. Modelo de Base de Datos Relacional

5.1 Motor y Base de Datos

Motor: Microsoft SQL Server. Base de datos: fundReserves.

Acceso desde consola:

```
sqlcmd -S localhost -U sa -P '<tu-password>' -d fundReserves
```

5.2 Diagrama de Relaciones

Tabla Padre	Tabla Hija	Relación	Descripción
sedes	accommodation	1 : N	Una sede tiene múltiples alojamientos
accommodation	rates	1 : N	Un alojamiento tiene tarifas por temporada
seasons	rates	1 : N	Una temporada aplica a varias tarifas
users	reserves	1 : N	Un usuario puede tener muchas reservas
accommodation	reserves	1 : N	Un alojamiento puede tener múltiples reservas
users	detalleReserve	1 : N	Un usuario puede tener varios detalles de reserva
sedes	detalleReserve	1 : N	Una sede puede tener múltiples detalles de reserva

5.3 Scripts SQL — Creación de Tablas

Tabla users

Almacena Administradores y Clientes. El campo rol diferencia el tipo de acceso.

```
CREATE TABLE users (
  idUser      INT          PRIMARY KEY IDENTITY(1,1),
  cedula      VARCHAR(20)  NOT NULL,
  password    NVARCHAR(255) NOT NULL,      -- almacenada con bcrypt.
  name        VARCHAR(100) NOT NULL,
  lastname    VARCHAR(100) NOT NULL,
  phone       VARCHAR(20),
  email       NVARCHAR(150) NOT NULL UNIQUE,
  rol         NVARCHAR(20)  NOT NULL,      -- 'Admin' | 'Customer'
  createdAt   DATETIME     DEFAULT GETDATE(),
  fechaNacimiento DATE,
  departamento VARCHAR(100),
  municipio   VARCHAR(100),
  barrio      VARCHAR(100),
  direccion   VARCHAR(200),
  preguntaSecreta VARCHAR(255),
  respuestaSecreta VARCHAR(255),
  autorizaCorreo BIT        DEFAULT 0,
  autorizaCelular BIT        DEFAULT 0,
  resetToken  NVARCHAR(200),
  resetTokenExpiration DATETIME
);
```

Tabla sedes

Registra las sedes recreativas y apartamentos del Fondo XYZ (Villeta, Fusagasugá, Chinchiná, Palmira, Santa Fe de Antioquia, Bogotá, Medellín, Santa Marta).

```
CREATE TABLE sedes (
  idSede      INT          PRIMARY KEY IDENTITY(1,1),
  name        VARCHAR(150) NOT NULL,
  city        VARCHAR(100) NOT NULL,
  region      VARCHAR(100),
  description  TEXT,
  maximumCapacity INT      NOT NULL,
  type        VARCHAR(50),  -- 'Recreativa' | 'Apartamento'
  priceLaundry DECIMAL(10,2) DEFAULT 0 -- tarifa lavandería (ej. $18.000)
);
```

Tabla accommodation

Representa cada alojamiento/habitación/apartamento dentro de una sede.

```
CREATE TABLE accommodation (
  idAccommodation INT          PRIMARY KEY IDENTITY(1,1),
  idSede          INT          NOT NULL
    REFERENCES sedes(idSede),
  name            VARCHAR(150) NOT NULL,
  number          INT          NOT NULL,
  maximumPerson   INT          NOT NULL,
  description     TEXT,
  state           VARCHAR(20)
);
```

Tabla seasons

Define las temporadas vigentes: Alta, Baja, Media, Especial Lunes-Jueves.

```
CREATE TABLE seasons (
  idSeason INT          PRIMARY KEY IDENTITY(1,1),
  type     VARCHAR(50)  NOT NULL,    -- 'Alta' | 'Baja' | 'Especial'
  name     VARCHAR(100) NOT NULL
);
```

```
);
```

Tabla rates

Almacena tarifas por noche según alojamiento, temporada y rango de personas.

```
CREATE TABLE rates (
    idPrice INT PRIMARY KEY IDENTITY(1,1),
    idAccommodation INT NOT NULL
        REFERENCES accommodation(idAccommodation),
    idSeason INT NOT NULL
        REFERENCES seasons(idSeason),
    minimumPerson INT NOT NULL,
    maximumPerson INT NOT NULL,
    priceNight DECIMAL(10,2) NOT NULL,
    pricePersonAdditional DECIMAL(10,2) DEFAULT 0
);
```

Tabla reserves

Registra las reservas individuales de alojamiento realizadas por los clientes.

```
CREATE TABLE reserves (
    idReserve INT PRIMARY KEY IDENTITY(1,1),
    idUser INT NOT NULL REFERENCES users(idUser),
    idAccommodation INT NOT NULL
        REFERENCES accommodation(idAccommodation),
    dateStart DATE NOT NULL,
    dateFinish DATE NOT NULL,
    numberPerson INT NOT NULL,
    totalCal DECIMAL(10,2) NOT NULL,
    state VARCHAR(30) DEFAULT 'Pendiente',
    -- 'Pendiente' | 'Confirmada' | 'Cancelada' | 'Rechazada'
    dateCreation DATETIME DEFAULT GETDATE(),
    serviceLaundry BIT DEFAULT 0
);
```

Tabla detalleReserve

Registra reservas a nivel de sede completa (múltiples habitaciones), incluyendo comprobante de pago.

```
CREATE TABLE detalleReserve (
    idDetalleReserve INT PRIMARY KEY IDENTITY(1,1),
    idUser INT NOT NULL REFERENCES users(idUser),
    idSede INT NOT NULL REFERENCES sedes(idSede),
    dateStart DATE NOT NULL,
    dateFinish DATE NOT NULL,
    numberPerson INT NOT NULL,
    numberRooms INT NOT NULL,
    totalCal DECIMAL(10,2) NOT NULL,
    paymentProof VARCHAR(500), -- ruta relativa: uploads/comprobantes/file.pdf
    createdAt DATETIME DEFAULT GETDATE()
);
```

5.4 Datos de Referencia — Sedes y Alojamientos del Fondo XYZ

Las siguientes sedes y alojamientos corresponden a los datos reales de la prueba técnica e inicializados mediante scripts INSERT en la base de datos:

Sede / Ciudad	Tipo	Alojamientos	Cap. Total
Villeta	Recreativa	8 habitaciones (cama doble + camarote + baño + TV + terraza)	32 pers.
El Placer – Fusagasugá	Recreativa	4 alojamientos (Nro 1-4) + 4 cabañas (Nro 5-8)	34 pers.
Gonzalo Morante – Chinchiná	Recreativa	Aloj. 1,2,4 + Cabaña tipo A (Aloj.3) + 2 tipo B (Aloj.5,6)	30 pers.
Tablones – Palmira	Recreativa	4 alojamientos (Nro 1-4)	24 pers.
Manguruma – Stafe Antioquia	Recreativa	3 alojamientos + bloque nuevo 8 alojamientos	46 pers.
Federman – Bogotá	Recreativa	4 habitaciones + zonas húmedas y gimnasio	—
Suramericana – Medellín	Apartamento	5 habitaciones (Hab 1 y 5 con baño privado)	—
El Rodadero – Santa Marta	Apartamento	Apto 202 (8p), Apto 301 (6p), Apto 401 (6p)	—

6. Procedimientos Almacenados

Se desarrollaron cuatro procedimientos almacenados en SQL Server para centralizar la lógica de disponibilidad y tarifas, optimizando el rendimiento y desacoplando la lógica de negocio crítica del código C#.

6.1 GetAvailableByDatesAndPersons — Disponibilidad por Fechas y Personas

Permite encontrar todos los alojamientos disponibles en un rango de fechas determinado, sin filtro por número de personas.

Parámetros de entrada:

- @dateStart DATE — Fecha de inicio del período.
- @dateFinish DATE — Fecha de fin del período.
- @number int —Número de personas por alojamiento.

Lógica: retorna todos los alojamientos cuyo idAccommodation no aparezca en ninguna reserva activa que se cruce con el rango solicitado.

```
CREATE PROCEDURE SP_GetAvailable
    @idSede INT,
    @dateStart DATE,
    @dateFinish DATE,
    @persons INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        a.idAccommodation,
        a.name,
        a.number,
        a.maximumPerson,
        a.description,
        a.state
```

```

FROM accommodation a
WHERE a.idSede = @idSede
      AND a.maximumPerson >= @persons
      AND a.state = 'Disponible'
      AND a.idAccommodation NOT IN (
          SELECT r.idAccommodation
          FROM reserves r
          WHERE r.dateStart < @dateFinish
                AND r.dateFinish > @dateStart
                AND r.state NOT IN ('Cancelada', 'Rechazada')
      );
END;

```

Llamada desde C# (Entity Framework / ADO.NET):

```

var result = await _context.Database
    .SqlQuery<AccommodationDto>($"
        EXEC SP_GetAvailable
            @idSede={idSede},
            @dateStart={dateStart},
            @dateFinish={dateFinish},
            @persons={persons}")
    .ToListAsync();

```

6.3 GetRates — Tarifas por Sitio, Temporada, Personas y Alojamiento

Permite consultar las tarifas aplicables de acuerdo al sitio (sede), la temporada, el número de personas y el alojamiento elegido.

- sede,
- rango de fechas,
- temporada,
- cantidad de personas,
- y disponibilidad del alojamiento.

Parámetros de entrada:

- @idAccommodation INT — Identificador del alojamiento elegido.
- @idSeason INT — Identificador de la temporada aplicable.
- @numberPerson INT — Número de personas para determinar tarifa base vs. adicional.

```

CREATE PROCEDURE SP_GetRates
    @idAccommodation INT,
    @idSeason          INT,
    @numberPerson      INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT
        r.idPrice,
        s.name           AS SedeNombre,
        s.city           AS Ciudad,
        a.name           AS AlojamientoNombre,
        a.number         AS NumeroAlojamiento,
        sn.name          AS Temporada,
        sn.type          AS TipoTemporada,
        r.minimumPerson,
        r.maximumPerson,
        r.priceNight      AS PrecioBaseNoche,
        r.pricePersonAdditional AS PrecioPersonaAdicional,

```

```

CASE
    WHEN @numberPerson > r.maximumPerson
    THEN (@numberPerson - r.maximumPerson) * r.pricePersonAdditional
    ELSE 0
END AS CargoAdicionalCalculado
FROM rates r
INNER JOIN accommodation a ON r.idAccommodation = a.idAccommodation
INNER JOIN sedes s ON a.idSede = s.idSede
INNER JOIN seasons sn ON r.idSeason = sn.idSeason
WHERE r.idAccommodation = @idAccommodation
    AND r.idSeason = @idSeason
    AND @numberPerson BETWEEN r.minimumPerson AND (r.maximumPerson + 10);
-- +10 para incluir personas adicionales sobre el rango máximo
END;

```

6.4 CalculateTotalRate — Cálculo de Tarifa Total a Cancelar

Calcula el valor total de la reserva considerando el alojamiento, número de personas, personas adicionales explícitas, temporadas dinámicas (Alta/Baja, Especial según día, festivos y recesos colombianos), noches y servicio de lavandería para los lugares que aplican.

Parámetros de entrada:

- **@idAccommodation** INT — Alojamiento elegido.
- **@idSeason** INT — Temporada aplicable al período.
- **@dateStart** DATE — Fecha de entrada.
- **@dateFinish** DATE — Fecha de salida.
- **@numberPerson** INT — Número de personas.
- **@extraPerson** INT — Número de persona Adicionales.
- **@numberRooms** INT — Número de habitaciones solicitadas.
- **@serviceLaundry** BIT — 1 si incluye servicio de lavandería, 0 si no.

```

CREATE PROCEDURE SP_CalculateTotalRate
    @idAccommodation INT,
    @dateStart DATE,
    @dateFinish DATE,
    @numberPerson INT,
    @extraPersons INT,
    @serviceLaundry BIT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @priceLaundry DECIMAL(10,2) = 0;
    DECLARE @numberOfNights INT;
    DECLARE @totalBase DECIMAL(10,2) = 0;
    DECLARE @totalAdditional DECIMAL(10,2) = 0;
    DECLARE @totalLaundry DECIMAL(10,2) = 0;
    DECLARE @totalFinal DECIMAL(10,2) = 0;
    DECLARE @nightsAlta INT = 0;
    DECLARE @nightsBaja INT = 0;
    DECLARE @priceNightAlta DECIMAL(10,2) = 0;
    DECLARE @priceNightBaja DECIMAL(10,2) = 0;
    DECLARE @priceAdditionalAlta DECIMAL(10,2) = 0;
    DECLARE @priceAdditionalBaja DECIMAL(10,2) = 0;

```

```

-- 1. Calcular número de noches total
SET @numberOfNights = DATEDIFF(DAY, @dateStart, @dateFinish);

-- 2. Desglosar noches por temporada (Alta / Baja)
-- Alta: viernes, sábado, domingo, festivos y recesos colombianos
-- Baja: lunes a jueves fuera de festivos y recesos
;WITH Nights AS (
    SELECT CAST(@dateStart AS DATE) AS NightDate
    UNION ALL
    SELECT DATEADD(DAY, 1, NightDate)
    FROM Nights
    WHERE DATEADD(DAY, 1, NightDate) < @dateFinish
)
SELECT
    @nightsAlta = SUM(CASE
        WHEN DATEPART(WEEKDAY, NightDate) IN (1, 6, 7) -- Dom, Vie, Sáb
        THEN 1 ELSE 0 END),
    @nightsBaja = SUM(CASE
        WHEN DATEPART(WEEKDAY, NightDate) NOT IN (1, 6, 7)
        THEN 1 ELSE 0 END)
FROM Nights
OPTION (MAXRECURSION 365);

-- Nota: los festivos y recesos oficiales colombianos se gestionan
-- en la capa de aplicación (frontend/backend). El SP aplica la
-- regla base de fin de semana; el total exacto lo produce la vista.

-- 3. Obtener tarifas por temporada para el alojamiento
-- usando @numberPerson para coincidir con el rango correcto
SELECT @priceNightAlta = r.priceNight,
    @priceAdditionalAlta = r.pricePersonAdditional
FROM rates r
INNER JOIN seasons s ON r.idSeason = s.idSeason
WHERE r.idAccommodation = @idAccommodation
    AND s.type = 'Alta'
    AND @numberPerson BETWEEN r.minimumPerson AND r.maximumPerson;

SELECT @priceNightBaja = r.priceNight,
    @priceAdditionalBaja = r.pricePersonAdditional
FROM rates r
INNER JOIN seasons s ON r.idSeason = s.idSeason
WHERE r.idAccommodation = @idAccommodation
    AND s.type = 'Baja'
    AND @numberPerson BETWEEN r.minimumPerson AND r.maximumPerson;

-- 4. Obtener precio de lavandería de la sede del alojamiento
SELECT @priceLaundry = s.priceLaundry
FROM accommodation a
INNER JOIN sedes s ON a.idSede = s.idSede
WHERE a.idAccommodation = @idAccommodation;

-- 5. Calcular componentes del total
SET @totalBase =
    (@priceNightAlta * @nightsAlta)
    + (@priceNightBaja * @nightsBaja);

-- Las personas adicionales (@extraPersons) se cobran por noche
-- según la tarifa adicional de cada temporada
SET @totalAdditional =
    (@extraPersons * @priceAdditionalAlta * @nightsAlta)
    + (@extraPersons * @priceAdditionalBaja * @nightsBaja);

SET @totalLaundry =
    CASE WHEN @serviceLaundry = 1 THEN @priceLaundry ELSE 0 END;

SET @totalFinal = @totalBase + @totalAdditional + @totalLaundry;

```

```
-- 6. Retornar desglose completo
SELECT
    @numberOfNights      AS NochesReservadas,
    @nightsAlta           AS NochesTemporadaAlta,
    @nightsBaja           AS NochesTemporadaBaja,
    @numberPerson         AS PersonasPrincipales,
    @extraPersons         AS PersonasAdicionales,
    @priceNightAlta       AS PrecioNocheAlta,
    @priceNightBaja       AS PrecioNocheBaja,
    @totalBase            AS TotalBase,
    @totalAdditional       AS TotalPersonasAdicionales,
    @totalLaundry         AS TotalLavanderia,
    @totalFinal           AS TotalAPagar;
END;
```

Fórmula general aplicada:

```
// TotalBase — precio por noche según temporada:
const subtotal = tarifa priceNight * n * qty;
// PrecioNoche × NochesDeEsaTemporada × CantidadAlojamientos
```

```
// TotalAdicional — personas extra por noche según temporada:
adicionalTotal += extraPersons * tarifaExtra pricePersonAdditional * n;
// PersonasAdicionales × PrecioAdicional × NochesDeEsaTemporada
```

```
// TotalLavandería:
grandTotal += laundryAmt;
// laundryAmt = withLaundry ? laundryPrice : 0
```

```
// TotalFinal:
grandTotal += laundryAmt;
// grandTotal = roomTotal + adicionalTotal + laundryAmt
```

7. Lógica de Disponibilidad y Reservas

7.1 Validación de Cruce de Fechas

¿Cómo sabe el sistema qué alojamientos están ocupados?

El principio central es la **detección de solapamiento de intervalos de fecha**. Dos reservas se cruzan si y solo si se cumple esta condición simultáneamente:

```
reserva_existente.dateStart < nueva.dateFinish
reserva_existente.dateFinish > nueva.dateStart
```

Esto cubre todos los casos problemáticos: reservas que empiezan en medio de otra, que la engloban completamente, o que terminan dentro de ella. Si cualquiera de las dos condiciones falla, los períodos no se tocan.

Importante: solo se bloquean reservas en estado “Pendiente” o “Pagado”. Las “Cancelada” se ignoran deliberadamente, devolviendo esa disponibilidad al pool. Lo puedes ver en `reservesController.cs`


```
var ocupados = _context.Reserves
    .Where(r => r.DateStart < dateFinish &&
        r.DateFinish > dateStart &&
        (r.State == "Pendiente" || r.State == "Pagado"))
    .Select(r => r.IdAccommodation)
    .Distinct()
    .ToList();
```

Flujo completo de una reserva desde el cliente

Paso 1: Consulta de disponibilidad (GET /api/reservations/available) El cliente envía sede, fechas, número de persona. El backend ejecuta la logica de cruce de fechas en memoria en EF Core (o vía el SP SPGetAvailable en SQL Server), y retorna solo los alojamientos cuyo idAccommodation no aparece entre los ocupados y cuya capacidad (maximumPerson) cubre al grupo. Cada alojamiento viene con sus tarifas aplicables al rango de personas.

Paso 2: Selecccion y cálculo de tarifa El cliente elige un alojamiento. El sistema invoca SPGetRates (o la logica equivalente) para obtener el precio por noche segun temporada, y luego SPCalculateTotalRate para el desglose final: noches de temporada alta, noches bajas, personas adicionales y lavanderia (solo si aplica).

Paso 3: Creación de la reserva (POST /api/reservations/create) Antes de persistir, el backend vuelve a validar el cruce de fechas (segunda barrera de seguridad), ya que pudo haber otra reserva creada en el tiempo que el usuario tardó en confirmar. Si ya está ocupado, retorna **409 Conflict**. Si está libre, guarda con estado "Pendiente" y la fecha de creación automática, aquí también se mira si el estado llega a estar "Cancelado" se podrá hacer la reserva.

Paso 4: Gestión por el administrador (PUT /api/reservations/updateState/{id}) El admin puede mover la reserva a "Pagado" o "Cancelado". Solo estos tres estados son validados, cualquier otro es rechazado con **400 Bad Request**.

Dos niveles de reserva en el sistema

El sistema maneja **dos tablas paralelas** con propósitos distintos:

Tabla	Controlador	¿Que representa?
reserves	reservesController	Reserva de un alojamiento específico (habitación/cabaña concreta)
detalleReserve	DetalleReserveController	Reserva de una sede completa (múltiples habitaciones a la vez), con comprobante de pago adjunto

DetalleReserve ademas maneja la subida del comprobante de pago en PDF: genera un nombre unico con Guid.NewGuid(), lo guarda en wwwroot/uploads/comprobantes/ y persiste solo la ruta relativa en la BD. El campo paymentProof arranca en null y se actualiza cuando el cliente sube el archivo, también se podrá reemplazar el archivo por si acaso el cliente se equivoca de archivo.

Explicación de los Procedimientos Almacenados

SPGetAvailable: ¿Qué alojamiento puede reservar?

Recibe sede, fechas y numeros de personas. Hace una sola consulta que excluye todos los idAccommodation que tengan reserva activa cruzada con el rango pedido (excluye estados 'Cancelado'). Adicionalmente filtra por maximumPerson >= @person y state = 'Disponible'. El resultado es la lista limpia de alojamientos reservables.

SPGetRates: ¿Cuánto cuesta por noche este alojamiento?

Recibe el alojamiento, la temporada y el número de personas. Hace join entre rates, accommodation, sedes, seasons para devolver el precio base por noche y el precio por persona adicional. Calcula en el mismo SELECT el cargo adicional si @numeberPerson supera el maximumPerson de la tarifa, usando CASE WHEN. El +10 en el BETWEEN del WHERE es el margen para incluir personas por encima del rango máximo de la tarifa.

SPCalculateTotalRate: ¿Cuánto pago en total?

Es el más complejo. Su lógica es:

1. Desglose de noches por temporada: Usa una CTE recursiva que genere una fila por cada noche del periodo y clasifica cada una como Alta (Viernes, Sábado, domingo) o Baja, Especial (lunes a jueves). Los festivos colombianos los maneja la capa de aplicación (frontend/backend C#).

2. **Obtención de tarifas por temporada:** Hace dos SELECTs independientes: uno para la tarifa Alta y otro para Baja o Especial, son filtrados por el idAccommodation y el rango de personas.

3. Cálculo del total con esta fórmula:

```
TotalBase      = (PrecioAlta × NochesAlta) + (PrecioBaja × NochesBaja)
TotalAdicional = PersonasExtra × (PrecioAdicionalAlta × NochesAlta
                                + PrecioAdicionalBaja × NochesBaja)
TotalLavandería = priceLaundry (solo si @serviceLaundry = 1)
TotalFinal      = TotalBase + TotalAdicional + TotalLavandería
```

4. **Retorna el desglose completo:** No solo el total, sino cada componente por separado (noches alta, noches baja, cargo adicional, lavandería), para que el frontend pueda mostrarle al cliente exactamente cómo se compone el precio.

Acumulación del total: por cada alojamiento seleccionado y por cada bloque de temporada:

```
subtotal = tarifa.priceNight × nochesDeEsaTemporada × cantidadAlojamientos
```

Las personas adicionales se calculan aparte sobre el primer alojamiento y se suman al gran total. La lavandería se agrega solo si el checkbox está marcado. El resultado exacto se guarda en la variable global **currentTotal** para usarlo sin pérdida de precisión al crear la reserva.

Las personas adicionales se calculan aparte sobre el primer alojamiento y se suman al gran total. La lavandería se agrega solo si el checkbox está marcado. El resultado exacto se guarda en la variable global `currentTotal` para usarlo sin pérdida de precisión al crear la reserva.

7.2 Variables Consideradas en el Cálculo

Variable	Fuente	Descripción
Cupo máximo por sede	<code>sedes.maximumCapacity</code>	Límite total de personas en la sede
Cupo por alojamiento	<code>accommodation.maximumPerson</code>	Personas máximas del alojamiento específico
Número de habitaciones	Entrada del usuario	Para calcular precio base total
Personas adicionales	<code>numberPerson – rates.maximumPerson</code>	Cargo adicional por persona extra
Temporada	<code>seasons.type</code>	Determina la tarifa base aplicable
Servicio lavandería	<code>sedes.priceLaundry + BIT</code>	Cargo adicional opcional en Santa Marta
Noches	<code>DATEDIFF(dateStart, dateFinish)</code>	Multiplicador del precio base

7.3 Flujo Completo de una Reserva

Paso	Acción	Componente
1	Cliente ingresa fechas y número de personas	Vista <code>reservesCustomer.cshtml</code>
2	Sistema invoca <code>GetAvailable</code>	<code>reservesController</code> → SQL Server
3	Resultados disponibles mostrados con tarifa estimada	Vista (lista dinámica JS)
4	Cliente selecciona alojamiento; sistema invoca <code>GetRates</code>	<code>reservesController</code>
5	Sistema invoca <code>CalculateTotalRate</code> para desglose de pago	<code>reservesController</code>
6	Cliente confirma; reserva persistida en tabla <code>reserves</code>	Entity Framework Core
7	Cliente recibe confirmación en <code>DashboardCustomer</code>	<code>DashboardCustomer.cshtml</code>
8	Admin puede cambiar estado (Confirmada / Cancelada)	<code>reservesController.UpdateState</code>

8. Configuración del Proyecto

8.1 appsettings.json

8.1 appsettings.json

Archivo de configuración principal de la aplicación.

Gestiona la cadena de conexión a SQL Server y los parámetros del servicio de correo SMTP utilizados para el envío de correos electrónicos.

Para el funcionamiento del envío de correos, es necesario configurar una aplicación en Gmail y generar una contraseña de aplicación.

Este correo será utilizado como remitente para los procesos de recuperación y cambio de contraseña de los usuarios.

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=fundReserves;
                          User Id=sa;Password=<tu-password>;
                          TrustServerCertificate=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "EmailSettings": {
    "From": "correo@gmail.com",
    "SmtpServer": "smtp.gmail.com",
    "Port": 587,
    "Username": "correo@gmail.com",
    "Password": "<app-password-google>"
  }
}
```

Sección	Descripción
ConnectionStrings.DefaultConnection	Cadena de conexión a SQL Server local con usuario sa y TrustServerCertificate para desarrollo
Logging	Nivel de log Information en general; Warning para los namespaces internos de ASP.NET Core
AllowedHosts	Permite peticiones desde cualquier host (*). En producción se debería restringir al dominio
EmailSettings	Configuración SMTP de Gmail (puerto 587 STARTTLS). Password = App Password de cuenta Google, no la contraseña real de la cuenta

8.2 Program.cs — Servicios y Pipeline

Punto de entrada de la aplicación. Registra todos los servicios en el contenedor de inyección de dependencias (DI) y configura el pipeline de middlewares:

```
// — Registro de servicios —————
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.Configure<EmailSettings>(
    builder.Configuration.GetSection("EmailSettings"));
builder.Services.AddTransient<IEmailService, EmailService>();

builder.Services.AddControllersWithViews()
    .AddJsonOptions(opt =>
        opt.JsonSerializerOptions.PropertyNamingPolicy =
            System.Text.Json.JsonNamingPolicy.CamelCase);

builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.IdleTimeout      = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly  = true;
    options.Cookie.IsEssential = true;
});

// — Pipeline de middlewares —————
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseSession();           // antes de UseAuthorization
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.MapControllers();
app.Run();
```

9. Seguridad

El sistema implementa los siguientes mecanismos de seguridad, diseñados para cumplir con los requerimientos de la prueba técnica:

Mecanismo	Implementación	Detalle
Autenticación	ASP.NET Session + Cookie HttpOnly	Sesión creada en login, destruida en logout. Timeout 30 min. Cookie no accesible desde JS
Hash de contraseñas	BCrypt	Las contraseñas nunca se almacenan en texto plano. Se genera un hash seguro en el registro y se compara en el login
Control por roles	Columna rol en users	'Admin' accede a panel de gestión; 'Customer' accede solo a reservas. Verificación manual en cada acción protegida
Recuperación de contraseña	Token GUID + SMTP	Se genera un resetToken único con expiración (resetTokenExpiration). El enlace incluye el token y solo es válido una vez

Mecanismo	Implementación	Detalle
HTTPS	UseHttpsRedirection	Todo el tráfico HTTP es redirigido a HTTPS en producción mediante middleware de ASP.NET Core
Protección archivos	uploads/comprobantes/	Los comprobantes subidos se almacenan fuera de wwwroot/css para evitar exposición directa. Acceso controlado por controlador
CamelCase JSON	JsonNamingPolicy.CamelCase	Evita exponer nombres de propiedades del modelo C# directamente en respuestas JSON del backend

10. Tecnologías, Librerías y Componentes

10.1 Backend

Tecnología / Librería	Versión	Uso en el proyecto
ASP.NET Core MVC	.NET 8	Framework principal: routing, controladores, vistas, pipeline HTTP
C#		Lenguaje principal: modelos, controladores, servicios, lógica de negocio
Entity Framework Core	8.x	ORM: mapeo objeto-relacional, DbContext, LINQ queries, migraciones
Microsoft SQL Server	17.0.4040.1	Motor de BD relacional: tablas, procedimientos almacenados, relaciones FK
System.Net.Mail (SMTP)	—	Envío de correos de recuperación de contraseña vía Gmail port 587
Razor Views	—	Motor de plantillas: genera HTML dinámico mezclando C# y marcado
Postman	—	Prueba manual de endpoints del backend durante desarrollo

10.2 Frontend

Tecnología	Uso en el proyecto
HTML5	Estructura semántica de todas las vistas Razor
CSS3	Estilos por módulo: login.css, dashboard.css, dashboardCustomer.css, reservesCustomer.css, site.css
JavaScript (Vanilla)	Cálculo dinámico de tarifas, llamadas fetch/JSON a endpoints del backend, validaciones cliente
Razor (.cshtml)	Vistas dinámicas: combina C# con HTML para renderizar datos del servidor en el cliente

10.3 Paquetes NuGet

Paquete NuGet	Uso
Microsoft.EntityFrameworkCore.SqlServer	Proveedor EF Core para SQL Server

Paquete NuGet	Uso
Microsoft.EntityFrameworkCore.Tools	Herramientas CLI: dotnet ef migrations, dotnet ef database update
Microsoft.AspNetCore.Session	Middleware de sesión del lado servidor
Microsoft.Extensions.Options.ConfigurationExtensions	Binding de EmailSettings desde appsettings.json

11. Instrucciones para Ejecutar el Proyecto

11.1 Requisitos Previos

Software	Versión mínima	Descarga
Visual Studio		visualstudio
.NET SDK	8.0	dotnet.microsoft.com/download
Microsoft SQL Server	16	microsoft.com/sql-server
Postman (opcional)	Cualquiera	postman.com

11.2 Pasos de Configuración

1. Descomprimir el archivo o clonar el repositorio :
git clone [git@github.com:jhoancamilorayomejia/fundReserves.git](https://github.com/jhoancamilorayomejia/fundReserves.git)
2. ingresar al archivo cd fundReserves, abrir el cmd y colocar code . y ejecutar.
3. Crear la base de datos en sqlcmd (CREATE DATABASE fundReserves;
4. Ejecutar el script SQL completo (tablas de users, sedes, accommodation, seasons, rates, reserves, detalleReserves)
5. Configuración appsettings.json con los valores locales (contraseña SQL Server y App Password de Gmail)
6. Restaurar paquetes NuGet: dotnet restore
7. Compilar el proyecto: dotnet build
8. Ejecutar la aplicación: dotnet run
9. Acceder al sistemas en el navegador: <https://localhost:5156>

importante si quieres ingresar al panel de Administrador, es nuevo hacer un insert con los datos requisitos, con el rol de Admin y con una contraseña encriptada con bcrypt que sepas.

11.3 Entregables del Proyecto

Entregable	Descripción	Formato
Script BD	Creación de tablas, procedimientos almacenados e INSERT de datos iniciales	script_fundReserves.sql
Proyecto VS	Código fuente completo del proyecto ASP.NET Core MVC	fundReserves.zip URL: github
Documento técnico	Este documento con arquitectura, BD, SPs, rutas y configuración	PDF

12. Conclusiones

El desarrollo del sistema fundReserves implementó una solución completa para la gestión de reservas del Fondo XYZ, cubriendo todos los requerimientos funcionales y técnicos de la prueba.

Requerimientos técnicos cubiertos:

- Estructura de BD relacional en SQL Server con 6 tablas normalizadas y relaciones FK correctamente definidas.
- 4 procedimientos almacenados: disponibilidad por fechas, disponibilidad por fechas y personas, consulta de tarifas y cálculo de tarifa total.
- Login seguro con sesiones ASP.NET Core, y hash bcrypt de contraseñas.
- Recuperación de contraseña vía SMTP Gmail con token de expiración.
- CRUD completo de usuarios, sedes, temporadas, tarifas, alojamientos y reservas, detalleReserves.
- Formulario de consulta de disponibilidad con invocación de procedimientos almacenados.
- Creación y consulta de reservas con cálculo automático de tarifa total.
- Programación en capas: Controllers, Models, Data, Services, Views, ViewModels.
- Patrón MVC con Razor Views y respuestas JSON para interactividad frontend.
- Entity Framework Core como ORM con ApplicationDbContext y migraciones.

La arquitectura multicapa adoptada facilita la escalabilidad, el mantenimiento y la incorporación de nuevas funcionalidades como pagos en línea o notificaciones push, sin impactar las capas existentes.